

Hannah Reflection

My contributions to the final project primarily consisted of the threads-based implementation, which I worked on with Grayson. To achieve this, we utilized our in-class time to iron out the high-level logic. We initially mapped out the exact state transitions, specifically how a cat's arrival needs to instantly trigger a flight response in the feeding mice, and designed the shared `SimState` structure in `simulation.h` together. Given some technical issues I faced on my own machine, along with simplifying collaboration while in the classroom, we decided to primarily use Grayson's machine for pushing code to GitHub. During the process, we would sit together, talk through the edge cases, write our respective blocks of code, and then immediately review them side-by-side. If I had any logic or code that needed to be combined with Grayson's or would potentially cause a conflict, I would then send what I wrote over to her so that it could be combined onto one machine before pushing. This pair-programming style kept our coding seamless and completely eliminated repeated occurrences of git merge conflicts.

While the state structure was a joint design effort, my primary individual coding responsibilities were implementing `mouse.c` and writing the setup and driver logic in `main.c`. Implementing the mouse lifecycle (`mouse.c`) required careful coordination because the rules dictate that mice must instantly vacate the bowls if a cat arrives. A standard, uninterrupted sleep interval would not work here. To address this, I implemented an incremental feeding strategy where the mouse thread sleeps in short 50ms bursts. At the end of each burst, it acquires the mutex lock and checks the `active_cats` counter. If a cat is present, the loop breaks, the thread logs the flight response, and it releases the bowl semaphore so the mouse can exit safely. Additionally, I added a safety check immediately after a mouse acquires its bowl semaphore; if a cat arrives in the brief window between the mouse clearing the condition variable and capturing the bowl, the mouse immediately aborts its feeding cycle. In `main.c`, I handled the core execution logic. This included parsing command-line parameters using `getopt`, initializing the POSIX named semaphores and condition variables, spawning the respective threads, and managing the teardown and unlinking sequences when the simulation is terminated by the user.

This project provided practical insight into how low-level synchronization primitives interact under contention. While I previously viewed mutexes and condition variables as relatively separate components, implementing the mouse logic required a clear understanding of how they function together, particularly how a thread waits on a condition variable while managing its associated guard lock. The assignment also highlighted the complexities of concurrency testing. Because race conditions rarely produce immediate compilation errors or straightforward segmentation faults, I relied heavily on analyzing the timestamped log lines to track down subtle timing issues. Verifying that the simulation could handle restrictive edge cases, such as a single bowl with high contention, without encountering a deadlock was an instructive part of the testing process.

Working with Grayson was highly efficient. Our communication was straightforward, and this helped us balance the development workload evenly and successfully work through problems alongside each other. Coordinating with the broader team in general, namely Anushka and

Vivian, was equally positive. Everyone maintained consistent communication, which kept our progress aligned and continually progressing.